

BỘ CÂU HỎI & TRẢ LỜI — BẢO VỆ ĐỒ ÁN

StoryLens

Nền tảng AI dịch & hỏi–đáp Manga theo ngữ cảnh
Chuẩn bị cho phần vấn đáp: từ tài liệu PA + sản phẩm chạy thật

Môn học CSC10011 – Công nghệ phần mềm cho hệ thống Trí tuệ Nhân tạo
GVHD GS.TS Nguyễn Văn Vũ, ThS. Trương Phước Lộc, ThS. Ngô Ngọc Đăng Khoa
Nhóm Nhóm 1 – StoryLens
Học kỳ Xuân 2026 · HCMUS

Contents

1	Tổng quan đề tài & Định vị	2
2	Yêu cầu & Use Case	2
3	Kiến trúc & Quyết định thiết kế	3
4	Pipeline AI & Mô hình ML	4
5	Hệ thống Hỏi-Đáp RAG	5
6	Quy trình, Quản lý dự án & Vai trò	6
7	Kiểm thử, CI/CD & Chất lượng	6
8	Bảo mật	7
9	Triển khai & Vận hành	7
10	Giám sát mô hình (MLOps Model Monitoring)	8
11	Khác biệt giữa tài liệu PA và sản phẩm hiện tại	9
12	Hạn chế, Lỗi & Hướng phát triển	10
13	Câu hỏi phản biện & đóng góp cá nhân	10

Cách dùng tài liệu. Mỗi câu là một tình huống thầy có thể hỏi khi bảo vệ, kèm gợi ý trả lời ngắn gọn, đúng trọng tâm. Người phụ trách mảng nào nên nắm chắc mảng đó (xem phân công trong file *Script_Thuyet_Trinh*). Đặc biệt lưu ý **Mục 10 — Giám sát mô hình (bổ sung mới)** và **Mục 11 — khác biệt giữa tài liệu PA và sản phẩm hiện tại**: đây là nhóm câu hỏi dễ bị hỏi nhất và cần trả lời trung thực theo tinh thần *phát triển lặp (iterative)* của RUP.

1. Tổng quan đề tài & Định vị

1. Đề tài của nhóm giải quyết vấn đề gì? [Tài liệu PA]

Trả lời. Cộng đồng đọc manga Việt Nam thiếu một nền tảng dịch tự động *hiểu ngữ cảnh* và có *hỏi–đáp thông minh*. Ba điểm đau: bản dịch fan-sub chậm và không đều; công cụ dịch máy (Google Lens) dịch rời rạc, mất xung hô và mạch truyện; và người đọc không có kênh hỏi ngay về nội dung. StoryLens giải quyết cả ba: dịch có ngữ cảnh cả trang, đọc overlay, và hỏi–đáp bằng RAG.

2. Vì sao chọn đề tài này? Tính thực tiễn ở đâu? [Tài liệu PA]

Trả lời. Manga là thị trường nội dung rất lớn ở Việt Nam nhưng lượng truyện gốc chưa dịch còn khổng lồ. Đề tài vừa có nhu cầu thật, vừa hội tụ nhiều kỹ thuật AI hiện đại (phát hiện đối tượng, OCR, inpainting, dịch máy có ngữ cảnh bằng LLM, và RAG) — phù hợp với môn Công nghệ phần mềm cho hệ thống AI, cho phép nhóm thực hành trọn vòng đời một sản phẩm AI thật.

3. Ai là người dùng mục tiêu? [Tài liệu PA]

Trả lời. Ba persona: (1) **sinh viên đại học** (18–25) đọc và tìm hiểu nội dung sâu; (2) **học sinh THPT** (15–18) cần giao diện dễ dùng, dịch nhanh; (3) **content creator** (25–35) làm video review, cần xử lý cả chương và sẵn sàng trả phí. Điểm chung: cần dịch chính xác và hỏi được về nội dung.

4. StoryLens khác gì Google Lens, DeepL/ChatGPT, hay fan-sub?

Trả lời. Google Lens dịch nhanh nhưng không hiểu ngữ cảnh; DeepL/ChatGPT phải thao tác thủ công, không hỏi–đáp thời gian thực trên chính nội dung; fan-sub thì chậm và không có AI. StoryLens là nền tảng *duy nhất* gộp cả bốn: dịch tự động, hiểu ngữ cảnh cả trang, đọc overlay, và hỏi–đáp RAG có trích nguồn.

5. Phạm vi (scope) sản phẩm tới đâu? Có gì ngoài phạm vi? [Sản phẩm hiện tại]

Trả lời. Trong phạm vi: dịch Nhật/Trung → Việt, đọc overlay, RAG Q&A, quản lý series/chương, thư viện công khai, cộng đồng, credit/thanh toán, quản trị. Ngoài phạm vi hiện tại: dịch video/anime, tự động hoá bản quyền, và ứng dụng di động native (đã có PWA, app native nằm ở hướng phát triển).

2. Yêu cầu & Use Case

6. Các yêu cầu chức năng chính là gì? [Tài liệu PA]

Trả lời. Chín nhóm F01–F09: xác thực & hồ sơ; upload ảnh; pipeline AI; reader & overlay; hỏi–đáp RAG; quản lý series/chương; lịch sử; credit & gói; và quản trị. Lỗi AI nằm ở F03 (pipeline) và F05 (RAG).

7. Actor của hệ thống là ai? [Tài liệu PA]

Trả lời. Trong đặc tả use case: **Người dùng thông thường** và **Content Creator** là hai actor chính; ngoài ra có **Admin** cho phần quản trị. Content Creator là người dùng nâng cao (batch cả chương, xuất bản, cần export).

8. Mô tả luồng chính của use case Upload & Dịch (UC01). [Tài liệu PA]

Trả lời. Người dùng kéo-thả ảnh (JPG/PNG, $\leq 10\text{MB}$) $\rightarrow$ backend kiểm định dạng, kích thước $\rightarrow$ gọi pipeline: YOLOv8 phát hiện bong bóng, manga-ocr đọc chữ Nhật $\rightarrow$ dịch có ngữ cảnh $\rightarrow$ lưu kết quả (bbox + text_vì) vào database và vector store $\rightarrow$ chuyển sang Reader. Hậu điều kiện: trang được lưu, bản dịch sẵn sàng.

9. UC01 có những luồng ngoại lệ nào? [Tài liệu PA]

Trả lời. Năm luồng thay thế: (1) sai định dạng $\rightarrow$ báo lỗi, huỷ; (2) quá 10MB $\rightarrow$ yêu cầu nén; (3) mất kết nối khi upload $\rightarrow$ nút “Thử lại”; (4) OCR không thấy text $\rightarrow$ vẫn lưu ảnh gốc, báo “không tìm thấy văn bản”; (5) LLM timeout $\rightarrow$ tự chuyển bản dịch dự phòng (M2M100/NLLB). Đây là điểm thể hiện nhóm có nghĩ tới độ bền của hệ thống.

10. Use case đọc overlay xử lý “text dịch quá dài” thế nào?

Trả lời. Theo đặc tả: thu nhỏ cỡ chữ; nếu vẫn tràn thì cắt bớt kèm “...” và tooltip đầy đủ. Trong sản phẩm thật, bước render dùng *tìm nhị phân cỡ chữ* để chọn cỡ lớn nhất còn vừa bounding box, đúng như tinh thần đặc tả.

11. Yêu cầu phi chức năng nổi bật là gì? [Tài liệu PA]

Trả lời. Hiệu năng: UI phản hồi $\leq 3\text{s}$, xử lý AI $\leq 30\text{s}/\text{trang}$, Q&A $\leq 10\text{s}$. Bảo mật: cookie HttpOnly, RBAC + RLS, rate-limit, CSP. Toàn vẹn: ledger credit chỉ ghi thêm; trang treo tự đánh dấu “failed” khi restart. Sẵn sàng: CDN cho frontend, backend co giãn, có xử lý cold start.

12. Vì sao trong đặc tả có use case “Add a product to cart”?

Trả lời. Đó là **văn bản mẫu còn sót** của template RUP (vốn cho ví dụ thương mại điện tử), không phải use case thật của StoryLens. Các use case thật của nhóm là Upload & Dịch, Đọc overlay, RAG Q&A, Batch upload, và Xem lịch sử. Nhóm xin ghi nhận và đã loại bỏ ở bản cập nhật.

3. Kiến trúc & Quyết định thiết kế

13. Mô tả kiến trúc tổng thể của hệ thống. [Sản phẩm hiện tại]

Trả lời. Kiến trúc *microservice* + *phân tầng*, ba dịch vụ triển khai độc lập: Frontend Next.js 16 (Vercel); Backend API FastAPI (Render, Docker, Singapore); AI Module FastAPI (HuggingFace Spaces, $\sim 4\text{GB}$). Trạng thái dùng chung đặt ở Supabase (PostgreSQL + pgvector + Auth + Storage). Google Gemini API lo phần dịch và sinh câu trả lời.

14. Vì sao tách AI Module thành dịch vụ riêng thay vì gộp vào backend? [Sản phẩm hiện tại]

Trả lời. Các mô hình (YOLOv8, manga-ocr, LaMa) nặng vài GB RAM. Backend chạy Render tier miễn phí chỉ 512MB, không thể tải nổi. Tách riêng giúp: (1) mỗi phần co giãn độc lập; (2) backend nhẹ, khởi động nhanh; (3) sự cố ở AI Module không kéo sập API chính. Đây là quyết định “isolate heavy ML workloads” ghi trong SAD.

15. Vì sao dùng pgvector thay vì một vector database riêng (ChromaDB/FAISS)?

Trả lời. Supabase đã cung cấp sẵn PostgreSQL, nên bật extension pgvector là có ngay tìm kiếm vector — **không phải vận hành thêm một dịch vụ**. Điều này giảm chi phí hạ tầng và độ phức tạp cho một MVP.

Tài liệu PA ban đầu dự kiến ChromaDB; nhóm đã đổi sang pgvector khi hiện thực (xem Mục 11).

16. Vì sao lưu token trong cookie HttpOnly thay vì localStorage? [\[Sản phẩm hiện tại\]](#)

Trả lời. Cookie HttpOnly **không đọc được từ JavaScript**, nên nếu có lỗ hổng XSS thì kẻ tấn công cũng không lấy được token — chống đánh cắp phiên. localStorage thì JS đọc được, rủi ro hơn. Đây là lý do dự án “không bao giờ lưu token ở localStorage”.

17. Backend dùng service-role key bỏ qua RLS --- vậy bảo mật dữ liệu thế nào? [\[Sản phẩm hiện tại\]](#)

Trả lời. RLS bảo vệ khi *truy cập trực tiếp* database bằng anon key. Backend dùng service-role để linh hoạt, và **tự kiểm tra quyền sở hữu trong code Python** trước mọi thao tác (mỗi series/page đều kiểm tra user_id). Hai lớp bổ sung cho nhau: RLS ở tầng DB, ownership check ở tầng ứng dụng.

18. Nêu một vài quyết định thiết kế quan trọng và lý do. [\[Sản phẩm hiện tại\]](#)

Trả lời. (1) AI Module riêng — cô lập tải ML nặng; (2) cookie HttpOnly — chống XSS; (3) pgvector — bớt một dịch vụ; (4) nhiều Gemini key xoay vòng — chống hết quota; (5) BoundedSemaphore cho pipeline — tránh tràn bộ nhớ trên tier free. Mỗi quyết định đều đánh đổi giữa chi phí, độ phức tạp và độ bền.

19. Các bảng dữ liệu chính? [\[Sản phẩm hiện tại\]](#)

Trả lời. profiles, manga_series, manga_chapters (cờ published_at cho thư viện), manga_pages, bubble_data (bbox, text, review_status), embeddings (pgvector), qa_history, credit_transactions, model_metrics (chỉ số chất lượng AI cho MLOps monitoring); cùng các bảng cộng đồng (forum, comments), thông báo, share links. Quan hệ: series → chapters → pages → bubbles/embeddings/model_metrics.

4. Pipeline AI & Mô hình ML

20. Mô tả chi tiết pipeline dịch, từng bước và công cụ. [\[Sản phẩm hiện tại\]](#)

Trả lời. Sáu bước: (1) **YOLOv8** phát hiện bong bóng thoại; (2) **manga-ocr** đọc chữ Nhật trong từng bbox; (3) **LaMa** inpaint xoá chữ gốc; (4) **Gemini 2.5 Flash** dịch có ngữ cảnh sang tiếng Việt (giữ xung hô); (5) **render** vẽ chữ Việt vào bbox (tìm nhị phân cỡ chữ); (6) sinh **embedding** câu, lưu pgvector. Chạy nền, giới hạn bằng semaphore, có huỷ hợp tác.

21. YOLOv8 huấn luyện trên dữ liệu nào, kết quả ra sao? [\[Sản phẩm hiện tại\]](#)

Trả lời. Dữ liệu **Manga109-s**: 956 ảnh test với 14.130 bounding box. Kết quả: **mAP@0.5 = 95,3%**, mAP@0.5:0.95 = 84,4%, Precision = 97,3%, Recall = 93,8% — đều vượt ngưỡng chấp nhận 85% nhóm đặt ra.

22. manga-ocr được fine-tune thế nào, đo bằng chỉ số gì? [\[Sản phẩm hiện tại\]](#)

Trả lời. Fine-tune trên Manga109-s (test 14.130 mẫu). Chỉ số: **CER (tỉ lệ lỗi ký tự) = 6,1%**, tức độ chính xác ký tự ~93,9%, và Exact-match 67,5%. CER là chuẩn đánh giá OCR: càng thấp càng tốt — 6,1% là mức tốt cho chữ manga nhiều biến thể.

23. Vì sao dịch bằng Gemini mà không train mô hình dịch riêng? [Sản phẩm hiện tại]

Trả lời. Dịch manga cần *hiểu ngữ cảnh, văn phong, xưng hô* — LLM như Gemini làm tốt việc này mà không cần tập dữ liệu song ngữ khổng lồ để tự train. Gemini nhận cả trang làm ngữ cảnh, nên bản dịch mạch lạc hơn dịch từng câu. Chi phí thấp (tier free + xoay vòng key) và triển khai nhanh.

24. Chất lượng dịch được đánh giá thế nào? [Sản phẩm hiện tại]

Trả lời. Bằng **human-in-the-loop**: người dùng/QC đánh giá và đạt trên 80% mức hài lòng; kèm cơ chế phản hồi thích/không thích và màn Studio QC cho phép duyệt/sửa từng bong bóng. Dịch máy khó chấm tự động tuyệt đối, nên vòng lặp con người là thước đo thực tế.

25. Nếu OCR đọc sai hoặc dịch sai thì hệ thống xử lý ra sao? [Sản phẩm hiện tại]

Trả lời. Vào **Studio QC** sửa tay từng bong bóng (approve/reject), có tìm-thay thế hàng loạt; mọi chỉnh sửa lưu *lịch sử hiệu đính*. Với bong bóng độ tin cậy thấp, reader hiển thị cảnh báo. Đây chính là cơ chế con-người-trong-vòng-lặp để bù cho giới hạn của mô hình.

26. Khi Gemini hết quota (429) thì sao? [Sản phẩm hiện tại]

Trả lời. Backend giữ một *pool nhiều API key* và **xoay vòng (round-robin)**; khi một key hết quota thì chuyển key khác, kèm throttle phía client và cache. Nếu cạn toàn bộ, hệ thống báo tính năng tạm không khả dụng thay vì lỗi thô. Admin có thể nạp thêm key và reload nóng.

27. Đánh giá tĩnh và động của mô hình khác nhau thế nào? Tái huấn luyện ra sao? [Sản phẩm hiện tại]

Trả lời. **Đánh giá tĩnh**: đo trên tập test cố định (mAP, CER) trước khi triển khai. **Đánh giá động**: theo dõi phản hồi thích/không thích và tỉ lệ hiệu đính khi chạy thật. Khi chất lượng tụt hoặc có domain mới, thu thập mẫu hiệu đính làm dữ liệu để fine-tune lại (chi tiết ở SAD mục 7).

28. Vì sao cần bước inpaint (LaMa)? Không xoá chữ gốc có được không? [Sản phẩm hiện tại]

Trả lời. Cần, vì phải *xoá chữ Nhật gốc* rồi mới vẽ chữ Việt lên đúng chỗ; nếu không, hai lớp chữ chồng nhau, không đọc được. LaMa inpaint tái tạo nền bong bóng mượt để chữ Việt hiển thị sạch.

5. Hệ thống Hỏi--Đáp RAG

29. RAG trong StoryLens hoạt động chính xác thế nào? [Sản phẩm hiện tại]

Trả lời. (1) Câu hỏi được vector hoá bằng gemini-embedding-001; (2) tìm cosine trong pgvector qua RPC match_embeddings, lấy top-k đoạn thoại liên quan; (3) ghép ngữ cảnh + câu hỏi đưa vào Gemini để sinh câu trả lời **kèm nguồn trích**. Nhờ “grounding” vào nội dung truyện nên hạn chế bịa (hallucinate).

30. Embedding bao nhiêu chiều, dùng mô hình gì?

Trả lời. Sản phẩm hiện tại dùng gemini-embedding-001 với **768 chiều**, lưu ở cột vector(768) (migration v7 — schema mới nhất là v8, bổ sung bằng model_metrics cho MLOps monitoring). Tài liệu/PA ban đầu ghi all-MiniLM-L6-v2 384 chiều — nhóm đã đổi khi hiện thực để đồng bộ chất lượng embedding với chính nhà cung cấp Gemini (xem Mục 11).

31. Làm sao RAG tránh trả lời bịa (hallucinate)? [Sản phẩm hiện tại]

Trả lời. Prompt yêu cầu Gemini **chỉ trả lời dựa trên các đoạn ngữ cảnh truy hồi được**; nếu không tìm thấy đoạn nào đủ ngưỡng tương đồng, hệ thống báo “chưa tìm thấy thông tin trong nội dung đã upload” thay vì bịa. Câu trả lời luôn kèm nguồn trích để người dùng kiểm chứng.

32. Mỗi câu hỏi tốn chi phí gì? [Sản phẩm hiện tại]

Trả lời. Mỗi câu trả lời *thành công* trừ **1 credit**. Gói free có 5 credit/ngày (reset 00:00 giờ Việt Nam); các gói trả phí có hạn mức tháng lớn hơn. 1 credit tương đương 1 ảnh dịch hoặc 1 câu Q&A.

6. Quy trình, Quản lý dự án & Vai trò

33. Nhóm dùng quy trình phát triển nào? [Tài liệu PA]

Trả lời. **RUP rút gọn kết hợp Scrum**: bốn pha Inception – Elaboration – Construction – Transition, mỗi sprint 2 tuần. Sản phẩm work-product theo RUP: project plan, vision, use case model, SAD, test plan/test case. Mỗi tuần họp scrum trả lời ba câu hỏi (đã làm gì / vướng gì / sắp làm gì).

34. Vai trò từng thành viên? [Tài liệu PA]

Trả lời. Năm vai trò, mỗi vai trò có người chính + dự phòng: **Trưởng nhóm/PM** (kế hoạch, phân công, báo cáo); **Business Analyst** (yêu cầu); **Designer** (kiến trúc, UI, SAD); **Implementer** (cần 3 người: code, unit test, review); **Tester** (test plan, test case, system test). Mọi thành viên đều dự họp và review yêu cầu.

35. Weekly report ghi những gì? [Tài liệu PA]

Trả lời. Theo mẫu scrum: công việc đã hoàn thành tuần qua, khó khăn đang gặp, và kế hoạch tuần tới, kèm tình trạng từng task theo người phụ trách. Nhóm nộp weekly report ở cả PA1, PA2, PA3.

36. Nhóm quản lý rủi ro thế nào? [Tài liệu PA]

Trả lời. Bảng rủi ro trong SDP: hết quota Gemini (xoay vòng key), OCR dưới ngưỡng (fine-tune + sửa tay), cold start HF/Render (keep-alive + retry), sai cấu hình RLS (review migration), và bản quyền (ToS + kiểm duyệt). Mỗi rủi ro có xác suất, mức ảnh hưởng và biện pháp giảm thiểu.

37. “Definition of Done” của nhóm là gì? [Tài liệu PA]

Trả lời. Một tính năng được coi là xong khi: code đã review và merge vào main; các test hiện có đều pass; có test tích hợp cho tính năng mới; đã kiểm thử UI (luồng chính + biên); và đã triển khai, xác minh trên môi trường thật.

7. Kiểm thử, CI/CD & Chất lượng

38. Chiến lược kiểm thử của nhóm? [Sản phẩm hiện tại]

Trả lời. Nhiều tầng: **unit/component** bằng Vitest + React Testing Library; **backend** bằng pytest + respx (mock Supabase/Gemini); **E2E** bằng Playwright (auth, home, protected pages, mobile, forum, edge-cases); và kiểm tra tĩnh (TypeScript strict, ESLint, type hints).

39. CI/CD được cấu hình ra sao? [Sản phẩm hiện tại]

Trả lời. GitHub Actions chạy **5 job song song**: typecheck, lint, production build, Playwright E2E, và backend pytest. Mọi pull request phải được review và CI phải xanh trước khi merge vào main; push main thì tự động deploy.

40. Làm sao test được phần AI vốn không định trước kết quả? [Sản phẩm hiện tại]

Trả lời. Với AI, nhóm mock lớp HTTP (respx) để test *luồng* pipeline mà không gọi thật; đo chất lượng mô hình bằng tập test tĩnh (mAP, CER); và dùng human-in-the-loop cho chất lượng dịch. Tách phần “logic phần mềm” (test tự động được) khỏi phần “chất lượng mô hình” (đo bằng metric + con người).

41. Test edge-case gồm những gì? [Sản phẩm hiện tại]

Trả lời. Bộ edge-cases kiểm: backend sập, bị rate-limit, phiên hết hạn, JSON hỏng, share link hết hạn (410), và trang 404 — đảm bảo UI xử lý lỗi mượt thay vì vỡ.

8. Bảo mật

42. Nêu các lớp bảo mật của hệ thống. [Sản phẩm hiện tại]

Trả lời. (1) Xác thực cookie HttpOnly; (2) phân quyền RBAC (user/admin) + RLS; (3) rate-limit theo endpoint (SlowAPI); (4) captcha Turnstile ở đăng ký/quên mật khẩu; (5) security headers + CSP chặt trên mọi response; (6) kiểm tra ảnh upload; (7) chống SSRF ở đường nhập từ nguồn ngoài.

43. Chống upload file độc hại thế nào? [Sản phẩm hiện tại]

Trả lời. Dịch vụ image_validation **giải mã lại bytes bằng Pillow verify()** để bác các payload giả mạo MIME (ví dụ file script đổi đuôi .jpg). Chỉ JPEG/PNG/WebP thật mới qua, kèm giới hạn kích thước.

44. SSRF là gì và nhóm chặn ra sao? [Sản phẩm hiện tại]

Trả lời. SSRF là khi kẻ tấn công lợi dụng chức năng “nhập từ URL” để ép server truy cập tài nguyên nội bộ. Ở đường import (scrape), scraper chỉ nhận URL thuộc **danh sách domain cho phép** (các site Madara/WordPress đã biết), nên không thể trỏ tới địa chỉ nội bộ tùy ý.

45. Dữ liệu người dùng được bảo vệ theo quyền riêng tư thế nào? [Sản phẩm hiện tại]

Trả lời. RLS trên các bảng dữ liệu người dùng; hỗ trợ **GDPR**: người dùng tự export hoặc xóa tài khoản; đổi email/mật khẩu tự phục vụ; token luôn trong cookie HttpOnly. Buckets ảnh gốc/thumbnaill để *private*, chỉ avatar public.

9. Triển khai & Vận hành

46. Hệ thống được triển khai ở đâu? [Sản phẩm hiện tại]

Trả lời. Bốn nền tảng: Frontend trên **Vercel** (CDN toàn cầu); Backend trên **Render** (Docker, Singapore); AI Module trên **HuggingFace Spaces**; dữ liệu trên **Supabase** (AWS). Tất cả tự động deploy khi push main.

47. Cold start là gì? Nhóm xử lý ra sao? [Sản phẩm hiện tại]

Trả lời. Tier miễn phí của Render và HF “ngủ” khi rảnh, lần gọi đầu phải khởi động lại (chậm 15–60s). Nhóm giảm bằng **keep-alive ping** (GitHub Actions) và **retry có backoff** phía frontend (8s, 15s). Đây là đánh đổi để chạy miễn phí.

48. Tiến trình xử lý cập nhật realtime bằng cách nào? [Sản phẩm hiện tại]

Trả lời. Qua **WebSocket** (/v1/ws/batch/...), có *fallback polling* khi WebSocket lỗi, và một **event bus** phát lại sự kiện đã lỡ khi client kết nối lại. Nhờ vậy thanh tiến trình “x/N trang” không bị mất cập nhật.

49. Giám sát và sao lưu ra sao? [Sản phẩm hiện tại]

Trả lời. Giám sát bằng **Sentry + OpenTelemetry**, health check /health và /health/deep (báo trạng thái Supabase, Gemini, AI Module). Sao lưu: Supabase PITR (WAL liên tục, giữ 7 ngày), pg_dump thủ công trước mỗi migration, và versioning cho storage.

10. Giám sát mô hình (MLOps Model Monitoring)**50. Hệ thống có giám sát chất lượng mô hình AI sau khi triển khai không? [Sản phẩm hiện tại]**

Trả lời. Có — đây là bổ sung mới nhất: **module Model Monitor** (services/model_monitor.py) tự động ghi lại chỉ số chất lượng cho *mỗi trang* ngay sau khi pipeline xử lý xong: số bong bóng phát hiện, độ tin cậy OCR trung bình, số bong bóng dịch thành công, độ trễ (ms), và tên bộ dịch dùng. Dữ liệu lưu ở bảng `model_metrics` (migration v8), hiển thị trên **Admin Dashboard** (/admin/model-monitor) dưới dạng KPI và biểu đồ theo ngày.

51. Cơ chế phát hiện model drift hoạt động thế nào? [Sản phẩm hiện tại]

Trả lời. So sánh **cửa sổ trượt 50 trang gần nhất** với 50 trang liền trước (baseline) trên ba chỉ số: độ tin cậy OCR trung bình, tỉ lệ trang có phát hiện bong bóng, và tỉ lệ dịch thành công. Có ngưỡng tuyệt đối (ví dụ OCR confidence dưới 55% → *critical*) và ngưỡng suy giảm tương đối (tụt hơn 5–10 điểm phần trăm so với baseline → *warning*). Trạng thái trả về: *ok* / *warning* / *critical* / *insufficient_data*, hiển thị ngay trên dashboard admin để cảnh báo sớm khi mô hình xuống cấp (data/concept drift) trên dữ liệu thật.

52. A/B testing mô hình được thực hiện ra sao? [Sản phẩm hiện tại]

Trả lời. Admin bật cờ `ab_test_variant` trong `app_settings` (off hoặc `experiment_50`). Khi bật, người dùng được chia **50/50 theo hash của user_id** vào nhóm *control* (cấu hình mặc định) hoặc *experiment* (cấu hình mô hình thay thế, ví dụ OCR/YOLO bản khác). Kết quả hai nhóm được so sánh qua endpoint `/admin/model-monitor/ab-results` — gộp theo tên bộ dịch (*translator*), so *success rate*, độ trễ và độ tin cậy OCR, rồi đề xuất phương án thắng.

53. Vì sao chọn tự xây module giám sát thay vì dùng công cụ MLOps có sẵn (MLflow, Evidently...)? [Sản phẩm hiện tại]

Trả lời. Ở quy mô đồ án/MVP, việc vận hành thêm một dịch vụ MLOps riêng (cần hạ tầng, lưu trữ, học cách dùng) không tương xứng với lợi ích. Nhóm chọn cách nhẹ: ghi metric trực tiếp vào Postgres đã có sẵn (Supabase), tính drift bằng thống kê đơn giản (so sánh trung bình cửa sổ trượt) ngay trong backend, và hiển thị trên dashboard admin đã có. Đây là bài học MLOps thực dụng: *giám sát tối thiểu khả thi (minimal viable monitoring)* vẫn tốt hơn không giám sát, và dễ nâng cấp lên công cụ chuyên dụng sau nếu cần.

54. Giám sát hạ tầng (Sentry/OpenTelemetry) khác gì giám sát mô hình AI (Model Monitor)?

Trả lời. Hai tầng bổ sung nhau, không thay thế nhau. **Giám sát hạ tầng** (Sentry, OpenTelemetry, /health) trả lời câu hỏi “hệ thống có đang chạy không” — lỗi 500, latency API, service có sống không. **Giám sát mô hình** (Model Monitor) trả lời câu hỏi khó hơn: “mô hình có đang *dịch/đọc đúng* không” — hệ thống có thể trả về HTTP 200 nhưng OCR đọc sai be bét hoặc YOLO bỏ sót bong bóng, mà giám sát hạ tầng không bao giờ phát hiện được vì không có exception nào xảy ra. Đây chính là khoảng trống (MLOps gap) mà môn học nhấn mạnh và module này được xây để lấp vào.

55. Ghi log metric có ảnh hưởng tới hiệu năng hay độ tin cậy pipeline dịch không? [\[Sản phẩm hiện tại\]](#)

Trả lời. Không. `log_pipeline_metrics()` chạy **best-effort**: bọc toàn bộ trong try/except, chỉ log cảnh báo nếu lỗi chứ **không bao giờ raise** — nếu ghi metric thất bại (ví dụ Supabase tạm gián đoạn), pipeline dịch chính vẫn hoàn tất bình thường. Đây là nguyên tắc thiết kế bắt buộc với mọi observability code: không được để tính năng phụ (giám sát) làm sập tính năng chính (dịch trang).

11. Khác biệt giữa tài liệu PA và sản phẩm hiện tại

Tinh thần trả lời: RUP là quy trình *lập*. Tài liệu PA là bản đặc tả ban đầu; khi hiện thực qua các sprint, một số lựa chọn kỹ thuật được **cải tiến có chủ đích** dựa trên ràng buộc thực tế (chi phí, hạ tầng, chất lượng). Trả lời trung thực + nêu *lý do* là điểm cộng, không phải điểm trừ.

56. PA ghi dùng ChromaDB + hybrid search (BM25 + semantic), nhưng sản phẩm dùng pgvector cosine. Vì sao đổi?

Trả lời. Khi hiện thực, nhóm đã có sẵn PostgreSQL trên Supabase, nên bật pgvector là có tìm kiếm vector mà **không phải vận hành thêm ChromaDB**. Với quy mô hiện tại, cosine trên pgvector đủ tốt và đơn giản hơn; bớt một dịch vụ giúp giảm chi phí và điểm hỏng. Hybrid BM25 có thể bổ sung sau nếu cần recall cao hơn.

57. PA nói dùng all-MiniLM-L6-v2 (384 chiều), nhưng code dùng gemini-embedding-001 (768 chiều). Vì sao?

Trả lời. Vì phần dịch và Q&A đã dùng hệ sinh thái Gemini, nên nhóm thống nhất luôn embedding bằng gemini-embedding-001 để chất lượng vector đồng bộ với mô hình sinh câu trả lời, và bớt một phụ thuộc nặng (sentence-transformers/torch) ở backend. Migration v7 tạo lại cột vector (768) cho khớp.

58. PA mô tả người dùng ẩn danh theo session_id và lưu 30 ngày, nhưng sản phẩm có tài khoản đầy đủ. Vì sao?

Trả lời. Khi mở rộng phạm vi (lich sử, thư viện công khai, credit, cộng đồng), nhóm cần định danh bền vững → chuyển sang **Supabase Auth** với cookie HttpOnly, kèm quên/đặt lại mật khẩu, Google OAuth và GDPR. Ý tưởng ẩn danh ban đầu vẫn còn ở phần đọc thư viện công khai (không cần đăng nhập).

59. PA từng ghi “Cloud-OCR”, nhưng sản phẩm dùng manga-ocr fine-tune. Vì sao?

Trả lời. manga-ocr chuyên cho chữ trong manga tiếng Nhật; sau khi fine-tune trên Manga109-s đạt CER 6,1% — tốt và **không phụ thuộc chi phí/giới hạn của Cloud Vision**. Vì thế nhóm chọn manga-ocr cho bản chạy thật.

60. PA nói fallback dịch bằng Google Translate/NLLB-200, còn hiện tại là M2M100/NLLB. Có mâu thuẫn không?

Trả lời. Ý tưởng bất biến là *có bản dịch dự phòng khi LLM lỗi/timeout*; nhóm chọn mô hình dịch mã nguồn mở (M2M100/NLLB) để chủ động và không phụ thuộc API trả phí. Đây là tinh chỉnh lựa chọn kỹ thuật, không đổi mục tiêu.

61. Phạm vi sản phẩm hiện tại rộng hơn nhiều so với 5 use case trong PA. Vì sao?

Trả lời. Đúng theo tinh thần RUP lặp: 5 use case là lõi ban đầu; qua các sprint Construction, nhóm bổ sung series/chương, credit & Stripe, thư viện, diễn đàn, thông báo, chia sẻ, gamification và admin — để sản phẩm đủ “production”. Các bổ sung này đều phục vụ cùng nhóm người dùng mục tiêu.

12. Hạn chế, Lỗi & Hướng phát triển

62. Hạn chế lớn nhất của hệ thống hiện tại? [Sản phẩm hiện tại]

Trả lời. Ba hạn chế chính: **cold start** do tier free; **quota Gemini** lúc cao điểm; và **OCR với chữ hiệu ứng/viết tay (SFX)** độ chính xác giảm. Ngoài ra bố cục chữ Việt dài đôi khi phải thu nhỏ/cắt, và thứ tự đọc phải-sang-trái ở vài bố cục phức tạp chưa tối ưu.

63. Lỗi đã biết (known defects) là gì? [Sản phẩm hiện tại]

Trả lời. Token phiên hết hạn giữa chừng đôi khi cần tải lại trang (đang làm auto-refresh); WebSocket có thể mất gói khi mạng chập chờn (đã có fallback polling); signed URL ảnh có thể hết hạn với phiên đọc rất dài (đã thêm retry); và nền động có thể tụt FPS trên mobile cũ (tắt được trong cài đặt).

64. Hướng phát triển tiếp theo? [Sản phẩm hiện tại]

Trả lời. Fine-tune thêm cho SFX/chữ viết tay và thêm cặp ngôn ngữ; cải thiện thứ tự đọc phải-sang-trái; nâng tier để giảm cold start + hàng đợi tác vụ chuyên dụng; cá nhân hoá gợi ý; bổ sung quy trình bản quyền/DMCA; và đóng gói ứng dụng di động.

65. Nếu scale lên hàng chục nghìn người dùng thì kiến trúc chịu được không? [Sản phẩm hiện tại]

Trả lời. Frontend đã ở CDN nên co giãn tốt. Backend đóng gói Docker có thể chạy nhiều instance; nút thắt là AI Module (GPU/CPU nặng) — cần chuyển sang tier trả phí, thêm hàng đợi (queue) và nhiều worker. Database Supabase có thể nâng gói và thêm index (ivfflat) cho pgvector. Tức kiến trúc đã tách đúng chỗ để scale từng phần.

13. Câu hỏi phản biện & đóng góp cá nhân

66. Vấn đề bản quyền manga thì sao? [Sản phẩm hiện tại]

Trả lời. StoryLens là **công cụ hỗ trợ đọc/dịch**, không phải nơi phát tán trái phép; có điều khoản sử dụng và công cụ kiểm duyệt trong admin. Người dùng chịu trách nhiệm với nội dung tự upload. Hướng phát triển sẽ bổ sung quy trình gỡ nội dung theo DMCA — đây là rủi ro đã ghi nhận trong SDP.

67. Chi phí vận hành hiện tại bao nhiêu? [Sản phẩm hiện tại]

Trả lời. Gần như **bằng 0** vì chạy trên tier miễn phí: Vercel Hobby, Render Free, HuggingFace Spaces free, Supabase Free, và Gemini free tier (xoay vòng key). Đánh đổi là cold start và giới hạn quota — chấp nhận

được cho phạm vi đồ án/MVP.

68. Phần nào của dự án khó nhất? Nhóm học được gì? [\[Sản phẩm hiện tại\]](#)

Trả lời. Khó nhất là **ghép pipeline AI nhiều bước chạy ổn định trên hạ tầng miễn phí hạn chế tài nguyên** — phải xử lý bất đồng bộ, giới hạn luồng, huỷ tác vụ, cold start và quota. Nhóm học được cách thiết kế hệ thống chịu lỗi và cân bằng giữa chất lượng AI với ràng buộc chi phí thực tế.

69. Vì sao không dùng một dịch vụ dịch manga có sẵn thay vì tự xây? [\[Sản phẩm hiện tại\]](#)

Trả lời. Các dịch vụ có sẵn thường không mở API, không cho tùy biến ngữ cảnh tiếng Việt, và **không có RAG Q&A**. Tự xây cho phép nhóm kiểm soát toàn bộ pipeline, tối ưu cho tiếng Việt, tích hợp cộng đồng/thư viện, và quan trọng là *học trọn vòng đời một hệ thống AND*.

70. Mỗi thành viên đóng góp cụ thể gì? [\[Tài liệu PA\]](#)

Trả lời. *(Mỗi bạn tự trả lời phần mình theo vai trò đã phân công.)* Gợi ý khung: nêu vai trò (PM/BA/Designer/Implementer/Tester), 2–3 hạng mục chính đã làm (ví dụ: hiện thực pipeline AI, dựng RAG, viết test E2E, thiết kế UI reader...), và một khó khăn đã vượt qua. Trả lời cụ thể, có dẫn chứng phần code/tài liệu mình phụ trách.

— Hết —
Chúc nhóm bảo vệ tốt!